# AJUSTES AL PROTOTIPO

**INTEGRANTES**:

- ALFREDO MALDONADO
- LUIS RIOS
- CRISTHIAN VELEZ

Fecha: 25 de FEB. de 26

Contenido

1. Objetivo General ..... 3

1.1 Objetivos Específicos ..... 3

2.1 Reporte de Optimización y Análisis Estadístico - Prototipo Segmentación K-Means v1.7 ..... 4

2.2 Ajustes de Prototipo ..... 4

2.3 Reporte de Análisis Estadístico ..... 5

2.4 Síntesis Reflexiva ..... 6

3. Bibliografía Seleccionada ..... 7

4. Anexos ..... 8

4.1 Repositorio del Proyecto ..... 8

4.2 Código Final del Prototipo Ajustado (v1.7) ..... 8

4.3 README v1.7 (Actualizado) ..... 10

4.4 Registro de Cambios (Bitácora - Changelog v1.7) ..... 10

# 1. Objetivo General

Desarrollar un prototipo de sistema de segmentación de clientes utilizando algoritmos de aprendizaje no supervisado (K-Means) para identificar perfiles de consumo diferenciados y optimizar la efectividad de las campañas de marketing en cadenas comerciales del sector retail, permitiendo una transición de estrategias masivas a acciones personalizadas basadas en datos transaccionales, bajo el marco legal de protección de datos en el Ecuador.

## 1.1 Objetivos Específicos

1. **Ejecutar el proceso de extracción, limpieza y preprocesamiento de datos** del historial transaccional en un periodo de dos semanas, calculando las métricas de Recencia, Frecuencia y Valor Monetario (RFM) para construir la base de conocimiento del modelo.y Determinar el número óptimo de clústeres *(k)* mediante la aplicación de técnicas de validación estadística como el Método del Codo y el Coeficiente de Silueta en un plazo de dos semanas, asegurando la cohesión interna y separación de los segmentos identificados.
2. **Implementar y entrenar el algoritmo K-Means** utilizando Python y Scikit-learn durante un ciclo de cinco semanas, logrando una clasificación de la base de clientes en segmentos accionables para el negocio (VIP, Leales, Ocasionales, En Riesgo O Similares).
3. **Construir un dashboard interactivo de visualización** mediante Streamlit en un periodo de tres semanas, que integre los resultados del modelo y permita a los gerentes de marketing explorar las características de cada segmento para el diseño de campañas. y Evaluar el impacto potencial del prototipo mediante la comparación de métricas históricas de conversión contra las proyecciones del modelo.

## 2.1 Reporte de Optimización y Análisis Estadístico - Prototipo Segmentación K-Means v1.7

## 2.2 Ajustes de Prototipo

El prototipo baseline (v1.6) presentaba deficiencias matemáticas graves en el tratamiento de los datos que invalidaban la eficacia del modelo K-Means. A continuación, se detallan los ajustes técnicos aplicados:

- **Mejora en el Preprocesamiento de Datos (Crítico):**
  - *Ajuste:* Se implementó una transformación logarítmica (np.log1p()) a las variables Recency, Frequency y Monetary (RFM) antes de aplicar StandardScaler(). Adicionalmente, se forzó la eliminación de transacciones con montos negativos o iguales a cero (devoluciones), ya que distorsionaban el logaritmo y el valor real del cliente.
  - *Por qué fue necesario:* El modelo v1.6 aplicaba escalado estándar directamente sobre datos crudos. Las variables transaccionales de retail tienen distribuciones altamente asimétricas (sesgo positivo extremo). Aplicar un K-Means a estos datos escalados linealmente provoca que los *outliers* (clientes que gastan cantidades masivas) arrastren los centroides, destruyendo la cohesión de los clústeres. Este fue el punto ciego más severo del baseline.
  - *Resultados esperados:* Clústeres más compactos y esféricos. Distribución equilibrada de clientes, evitando la generación de un "segmento basura" formado por dos o tres valores atípicos.
- **Ajuste de Hiperparámetros:**
  - *Ajuste:* Se configuró explícitamente init='k-means++' y n_init=10 en la instancia del algoritmo.
  - *Por qué fue necesario:* En v1.6 se omitieron los parámetros de inicialización, confiando en los valores por defecto que pueden converger a mínimos locales subóptimos. Esto asegura la convergencia matemática del modelo.

- **Selección o ingeniería de características:**
  - *Ajuste:* No se agregaron nuevas características (se mantiene RFM).
  - *Por qué:* Las métricas RFM son suficientes para este caso de uso comercial. Añadir características como *Tenure* (RFMT) en un historial transaccional de corto plazo introducía ruido sin aportar varianza explicativa útil.
- **Modificación de técnicas de modelado:**
  - *Ajuste:* Se mantiene K-Means. No se reemplazó por DBSCAN o agrupamiento jerárquico.
  - *Por qué:* Para segmentación de marketing (RFM), la partición dura y esférica de K-Means es la más interpretable matemáticamente para asignar las etiquetas solicitadas (VIP, Leales, etc.). La transformación logarítmica corrige la sensibilidad del modelo, haciendo innecesario el cambio de algoritmo.

## 2.3 Reporte de Análisis Estadístico

- **Técnica de validación aplicada:**
  - Se implementó la evaluación de la estabilidad del clúster mediante iteración y el cálculo combinado del Coeficiente de Silueta **y el** Índice de Davies-Bouldin**.** No se utilizó validación cruzada tradicional (K-Fold), ya que el aprendizaje no supervisado carece de etiquetas *ground-truth*. En su lugar, se evaluó la robustez de los centroides frente a variaciones de *k*.
- **Comparación entre el modelo baseline (v1.6) y ajustado (v1.7):**
  - **Baseline v1.6:** Mostraba alta susceptibilidad a *outliers*. Los centroides "VIP" se basaban únicamente en valores monetarios extremos, agrupando erróneamente en "Leales" a clientes que matemáticamente no presentaban cohesión de frecuencia.

- **Ajustado v1.7 (Log-RFM):** Normaliza el espacio de características. La distancia euclidiana calculada por el K-Means ahora representa variaciones relativas (porcentuales) y no absolutas. Los clústeres resultantes tienen distribuciones de volumen (conteo de clientes) mucho más lógicas para el negocio.

- **Métricas estadísticas utilizadas:**
  - **Inercia (Sum of Squared Errors):** Mide la compacidad interna.
  - **Coeficiente de Silueta:** Mide la distancia inter-clúster vs intra-clúster (rango de -1 a 1). En la v1.7 el score promedio aumenta debido a la eliminación del ruido asimétrico.
  - **Índice de Davies-Bouldin:** Añadido para medir la ratio de dispersión interna respecto a la separación. Valores más bajos indican un modelo superior.
- **Análisis de la variabilidad e incertidumbre:**
  - En v1.6, mínimas variaciones en el dataset (añadir una compra atípica) desplazaban el centroide VIP hasta en un 40%.
  - En v1.7, al aplicar la función logarítmica, la varianza intracluster de la variable *Monetary* se estabiliza. La incertidumbre sobre la clasificación en la frontera de decisión (ej. Leal vs VIP) se reduce drásticamente.
- **Interpretación de los hallazgos:**
  - La corrección de la asimetría demuestra que los "clientes VIP" no solo son los que gastan sumas exorbitantes una vez (outliers de *Monetary*), sino aquellos cuya recencia y frecuencia mantienen una relación proporcionalmente alta. El modelo ahora refleja una partición real de comportamiento de consumo y no simples cortes estadísticos sesgados.

## 2.4 Síntesis Reflexiva

- **Decisiones técnicas más relevantes:**
  - Descartar el escalado simple y forzar la transformación logarítmica (log1p). Escalar asimetrías severas es un error de

diseño básico que compromete la distancia euclidiana de K-Means.

  - Mantener las etiquetas de negocio dinámicas (Puntuación = M_z + F_z - R_z), pero ahora basadas en los centroides log-transformados, lo cual otorga validez matemática a la nomenclatura de marketing.

- **Riesgos técnicos identificados y mitigados:**
  - *Riesgo:* Valores negativos o nulos en montos monetarios (devoluciones) provocan valores NaN o -inf al aplicar logaritmos.
  - *Mitigación:* Se implementó un filtrado matricial estricto df = df[df[amount_col] > 0] previo al cálculo RFM.

## 3. Bibliografía Seleccionada


1. Paramita, A. S., & Hariguna, T. (2024). Comparative Studies on Clustering Algorithms K-Means and DBSCAN. Journal of Digital Market and Digital Currency. [1]
2. Gan, W., Zhou, P., Wan, S., & Zeng, J. (2025). RFMI-based Customer Segmentation with K-means. 2024 IEEE International Conference on Big Data. [2]
3. John, J. M., Shobayo, O., & Ogunleye, B. (2025). The RFM-A Model: Combining Behavioral and Demographic Data for Segmentation. The Journal of Supercomputing. [3]
4. Srivastava, P., & Saxena, D. S. (2025). Enhanced Customer: A Comparative Analysis of Leading Clustering Algorithms. International Journal of Novel Research and Development, 10(4). [4]
5. Nugroho, B. I., Rafhina, A., & Ananda, P. S. (2024). Customer segmentation in sales transaction data using k-means clustering algorithm. Journal of Intelligent Decision Support System, 7(2). [5]
6. Hidayat, T., Ruldeviyani, A., & Aditama, A. R. (2024). Optimal Use of Customer Segmentation using RFM and K-means. Scientific Journal of Informatics, 11(1). [6]
7. Lai, M. T. (2025). Comparison of K-Means and DBSCAN Algorithms for Customer Segmentation. ResearchGate. [7]
8. Lewaaelhamd, I. (2024). Customer segmentation using machine learning model: An application of RFM analysis. Journal of Data Science and Intelligent Systems, 2, 29–36. [8]

9. Nithyashree, R. (2024). Decoding Decision Support Systems: Empowering Malls with Customer Segmentation. Medium Data Science. [9]
10. Ogunleye, B. (2024). An Exploration of Clustering Algorithms for Customer Segmentation in the UK Retail Market. Analytics, 2(4), 809-823. [10]

# 4. Anexos

## 4.1 Repositorio del Proyecto

**URL Github**: https://github.com/lukito3192/PROYECTO-MIA-BETA.git

## 4.2 Código Final del Prototipo Ajustado (v1.7)

Se presenta el código que se agregará para incluir en app.py. el código se encuentra en el repositorio con sus respectivos controles de cambios por versionamiento.

```
import pandas as pd
import numpy as np
import streamlit as st
import plotly.express as px
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score

pd.set_option("styler.render.max_elements", 5000000)

def load_dataset(uploaded_file):
    if uploaded_file.name.endswith('.csv'):
        try:
            return pd.read_csv(uploaded_file, low_memory=False)
        except Exception:
            uploaded_file.seek(0)
            return pd.read_csv(uploaded_file, sep=';',
on_bad_lines='skip', low_memory=False)
    elif uploaded_file.name.endswith('.xlsx'):
        return pd.read_excel(uploaded_file)
    else:
        st.error("Formato no soportado")
        return None

def calculate_rfm(df, customer_col, date_col, amount_col):
    df[date_col] = pd.to_datetime(df[date_col], dayfirst=True,
format='mixed', errors='coerce')
    df[amount_col] = pd.to_numeric(df[amount_col], errors='coerce')

    # MITIGACIÓN RIESGO: Filtrar montos menores o iguales a cero
previos al logaritmo
    df = df.dropna(subset=[amount_col, date_col])
    df = df[df[amount_col] > 0]

    reference_date = df[date_col].max() + pd.Timedelta(days=1)

    rfm = df.groupby(customer_col).agg({
```

```
        date_col: lambda x: (reference_date - x.max()).days,
        customer_col: 'count',
        amount_col: 'sum'
    })
    rfm.columns = ['Recency', 'Frequency', 'Monetary']
    return rfm.reset_index()

def scale_features(rfm_df):
    # AJUSTE CRÍTICO PREPROCESAMIENTO: Transformación Logarítmica para
asimetría
    rfm_log = pd.DataFrame()
    rfm_log['Recency'] = np.log1p(rfm_df['Recency'])
    rfm_log['Frequency'] = np.log1p(rfm_df['Frequency'])
    rfm_log['Monetary'] = np.log1p(rfm_df['Monetary'])

    scaler = StandardScaler()
    scaled = scaler.fit_transform(rfm_log)
    return scaled

def calculate_elbow_silhouette(data, max_k=10):
    silhouette = []
    db_scores = []
    for k in range(2, max_k+1):
        # AJUSTE HIPERPARÁMETROS: n_init explícito
        model = KMeans(n_clusters=k, init='k-means++', n_init=10,
random_state=42)
        labels = model.fit_predict(data)
        silhouette.append(silhouette_score(data, labels))
        db_scores.append(davies_bouldin_score(data, labels))
    return silhouette, db_scores

def train_kmeans(data, k):
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10,
random_state=42)
    labels = kmeans.fit_predict(data)
    return kmeans, labels

def assign_dynamic_segments(rfm):
    # Uso logaritmo interno para evaluar los centroides sin distorsión
    rfm_log = rfm.copy()
    rfm_log[['Recency', 'Frequency', 'Monetary']] =
np.log1p(rfm[['Recency', 'Frequency', 'Monetary']])
    centroids = rfm_log.groupby('Cluster')[['Recency', 'Frequency',
'Monetary']].mean()

    scaler = StandardScaler()
    scaled_centroids = pd.DataFrame(
        scaler.fit_transform(centroids), columns=['R_z', 'F_z',
'M_z'], index=centroids.index
    )
    scaled_centroids['Score'] = scaled_centroids['M_z'] +
scaled_centroids['F_z'] - scaled_centroids['R_z']
    scaled_centroids = scaled_centroids.sort_values('Score',
ascending=False)

    cluster_to_label = {}
    for idx, (cluster_id, row) in
enumerate(scaled_centroids.iterrows()):
        if idx == 0: cluster_to_label[cluster_id] = 'VIP'
        elif idx == len(scaled_centroids) - 1:
cluster_to_label[cluster_id] = 'Perdidos'
```

```
        else:
            if row['R_z'] > 0: cluster_to_label[cluster_id] = 'En
Riesgo'
            else: cluster_to_label[cluster_id] = 'Leales'

    rfm['Segmento'] = rfm['Cluster'].map(cluster_to_label)
    return rfm

# [LA INTERFAZ DE STREAMLIT SE MANTIENE COMO EN LA V1.6 CON LA ADICIÓN
DE DAVIES-BOULDIN]
```

## 4.3 README v1.7 (Actualizado)

**Cambios principales a incorporar en el README de GitHub:**

1. **Modelado Robusto:** El sistema ahora emplea transformaciones logarítmicas (np.log1p) para normalizar el sesgo positivo del historial transaccional, garantizando centroides matemáticamente precisos.
2. **Nuevas Métricas:** Implementación del Índice de Davies-Bouldin para la evaluación combinada de validez interna de los clústeres.
3. **Filtros de Datos:** Prevención de colapsos algorítmicos mediante filtrado automático de montos <= 0.

## 4.4 Registro de Cambios (Bitácora - Changelog v1.7)

Cambios a incorporar en la bitácora:

**[v1.7] - Corrección de asimetría y estabilidad de K-Means (Versión Final)**

- **Acción:** Aplicación de np.log1p en preprocesamiento. Inclusión de davies_bouldin_score. Refactorización de K-Means con init='k-means++' y n_init=10. Filtrado de retornos (montos < 0).
- **Problema resuelto (Crítico de la v1.6):** La estandarización lineal de variables no normales destruía la varianza local, generando clústeres dominados por clientes atípicos (outliers) y haciendo inexactas las etiquetas de negocio.